

Reconnaissance — tools useful for this SSH lab

1) nmap

Purpose: Find open ports (like SSH), get service/version info and run small service scripts.

Install (Kali):

sudo apt update

sudo apt install -y nmap

```
(jadhav@Kali)-[~]
$ sudo apt install -y nmap
[sudo] password for jadhav:
nmap is already the newest version (7.95+dfsg-3kali1).
The following packages were automatically installed and are no longer required:
  criu      libintl-perl      libmongoc-1.0-0t64      libsort-naturally-perl      tini
  libbson-1.0-0t64  libintl-xs-perl      libmongocrypt0          needrestart
  libcompel1      libmodule-find-perl  libproc-processtable-perl  python3-pycru
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1303
(jadhav@Kali)-[~]
$
```

Example A — basic SSH port + service/version

`nmap -sS -Pn -p 22 -sV 172.16.209.85`

Options explained:

- `-sS` : TCP SYN (half-open) scan — fast, common.
- `-Pn` : skip host discovery (treat host as up) — useful in VMs with ICMP blocked.
- `-p 22` : scan port 22 only (target SSH).
- `-sV` : try to detect service/version running on the port.

What you'll see (scenario):

- Console: `22/tcp open ssh` and possible version string (e.g., `OpenSSH 8.x`).

```
(jadhav@Kali)-[~]
$ nmap -sS -Pn -p 22 -sV 172.16.209.85
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-11 14:08 IST
Nmap scan report for 172.16.209.85
Host is up (0.0030s latency).

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 10.0p2 Debian 7~bpo12+1 (protocol 2.0)
MAC Address: 08:00:27:E7:D7:3F (PCS Systemtechnik/Oracle VirtualBox virtual NI
C)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nm
ap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 0.89 seconds
(jadhav@Kali)-[~]
$
```

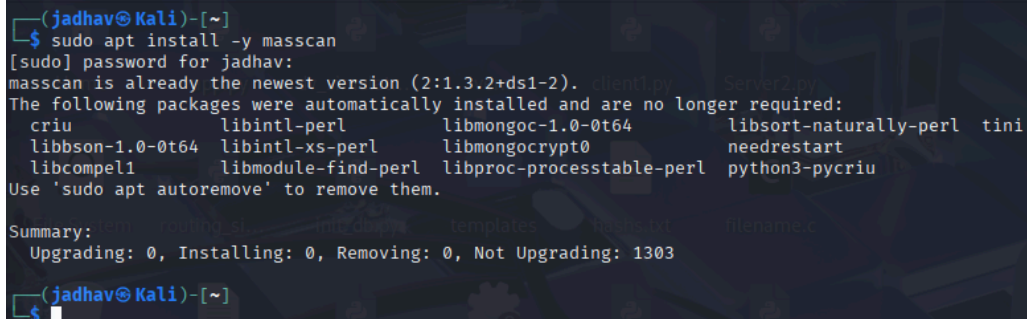
2) **masscan** — very fast port sweeper

Purpose: Sweep lots of IPs/ports quickly. Good for quick discovery in a subnet.

Install (Kali):

sudo apt update

sudo apt install -y masscan



```
(jadhav@Kali)-[~]
$ sudo apt install -y masscan
[sudo] password for jadhav:
masscan is already the newest version (2:1.3.2+ds1-2).
The following packages were automatically installed and are no longer required:
  criu libbison-1.0-0t64 libcompel1 libintl-perl libintl-xs-perl libmongocrypt0
  libmodule-find-perl libproc-processtable-perl libsort-naturally-perl tini
  needrestart python3-pycru
Use 'sudo apt autoremove' to remove them.
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1303
(jadhav@Kali)-[~]
$
```

single IP, port 22

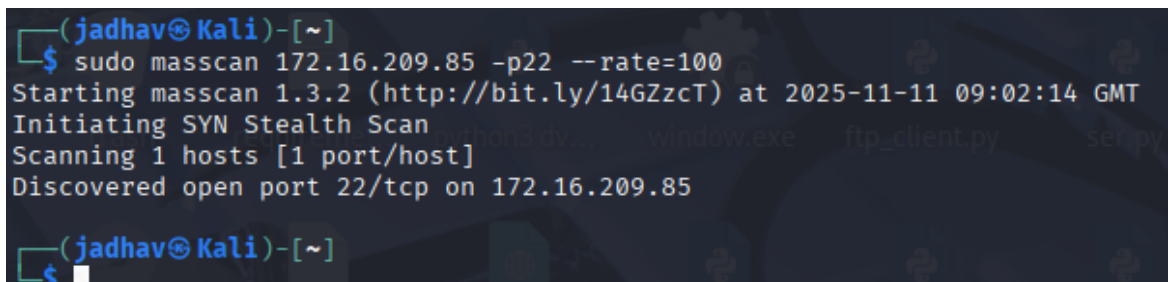
sudo masscan 172.16.209.85 -p22 --rate=1000

Options explained:

- **172.16.209.85** : target IP.
- **-p22** : port 22 only.
- **--rate=1000** : packets per second; controls scan speed (lower is gentler).

What you'll see (scenario):

- Console: **Discovered open port 22 on 172.16.209.85** (quick).



```
(jadhav@Kali)-[~]
$ sudo masscan 172.16.209.85 -p22 --rate=100
Starting masscan 1.3.2 (http://bit.ly/14GZzcT) at 2025-11-11 09:02:14 GMT
Initiating SYN Stealth Scan
Scanning 1 hosts [1 port/host]
Discovered open port 22/tcp on 172.16.209.85
(jadhav@Kali)-[~]
$
```

scan your VM subnet

sudo masscan 172.16.192.0/19 -p22 --rate=500

Options explained:

- **172.16.192.0/19** : network range that includes your Parrot IP.
- **--rate=500** : slower, safer in small VM networks.

Notes / safety: masscan is very noisy. Keep **--rate** low in VMs. It is best used to generate recon traffic for detections.

```
(jadhav@Kali)-[~]
$ sudo masscan 172.16.192.0/19 -p22 --rate=500
Starting masscan 1.3.2 (http://bit.ly/14GZzcT) at 2025-11-11 09:04:59 GMT
Initiating SYN Stealth Scan
Scanning 8192 hosts [1 port/host]
Discovered open port 22/tcp on 172.16.198.212
Discovered open port 22/tcp on 172.16.211.255
Discovered open port 22/tcp on 172.16.200.91
Discovered open port 22/tcp on 172.16.214.142
Discovered open port 22/tcp on 172.16.214.72
Discovered open port 22/tcp on 172.16.202.170
Discovered open port 22/tcp on 172.16.214.103
Discovered open port 22/tcp on 172.16.207.62
Discovered open port 22/tcp on 172.16.203.61
Discovered open port 22/tcp on 172.16.206.0
Discovered open port 22/tcp on 172.16.216.85
Discovered open port 22/tcp on 172.16.214.140
Discovered open port 22/tcp on 172.16.214.124
Discovered open port 22/tcp on 172.16.198.90
Discovered open port 22/tcp on 172.16.222.82
Discovered open port 22/tcp on 172.16.218.42
Discovered open port 22/tcp on 172.16.214.110
Discovered open port 22/tcp on 172.16.197.220
Discovered open port 22/tcp on 172.16.194.83
Discovered open port 22/tcp on 172.16.203.212
Discovered open port 22/tcp on 172.16.200.250
Discovered open port 22/tcp on 172.16.192.97
Discovered open port 22/tcp on 172.16.202.181
Discovered open port 22/tcp on 172.16.217.144
Discovered open port 22/tcp on 172.16.196.232
Discovered open port 22/tcp on 172.16.199.169
Discovered open port 22/tcp on 172.16.194.84
Discovered open port 22/tcp on 172.16.209.18
Discovered open port 22/tcp on 172.16.194.93
Discovered open port 22/tcp on 172.16.203.21
Discovered open port 22/tcp on 172.16.221.99
```

3) **rustscan** — fast probe then nmap detail

Purpose: Fast initial probe; automatically hands results to **nmap** for deeper info.

Install (Kali):

RustScan might not be in older repos — use the prebuilt .deb or cargo. Quick apt attempt:

sudo apt update

sudo apt install -y rustscan (if this not work then download form github)

Linke : <https://github.com/bee-san/RustScan/tree/2.4.1>

Else you can run this command :

```
(jadhav@Kali)-[~/Downloads/RustScan-2.4.1]
$ # 1. Install Rust (if not already)
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh

# 2. Load cargo environment
source $HOME/.cargo/env

# 3. Go to the RustScan folder
cd ~/Downloads/RustScan-2.4.1

# 4. Build RustScan
cargo build --release

# 5. Move the binary to system path
sudo cp target/release/rustscan /usr/local/bin/

# 6. Verify installation
rustscan -V
```

Example — probe then nmap

rustscan -a 172.16.209.85 -p 22 -- -sV -Pn

Options explained:

- **-a 172.16.209.85** : target IP.
- **-p 22** : probe port 22.
- **--** : end of rustscan options; following options are passed to **nmap**.
- **-sV -Pn** : same nmap options as earlier (service/version + treat host as up).

What you'll see (scenario):

- RustScan output: quick open/closed port discovery.

- clear

Notes: RustScan is convenience: fast + automatically runs nmap for details.

```
(jadhav@Kali)-[~/Downloads/RustScan-2.4.1]
$ rustscan -a 172.16.209.85 -p 22 -- -sV -Pn
rustscan releases latest
The Modern Day Port Scanner.
: http://discord.skerritt.blog
: https://github.com/RustScan/RustScan :
Manual for more information
Oday was here ♥

[~] The config file is expected to be at "/home/jadhav/.rustscan.toml"
[!] File limit is lower than default batch size. Consider upping with --ulimit. May cause harm to sensitive servers
[!] Your file limit is very small, which negatively impacts RustScan's speed. Use the Docker image, or up the Ulimit
with '--ulimit 5000'.
Open 172.16.209.85:22
[~] Starting Script(s)
[>] Running script "nmap -vvv -p {{port}} --{{ipversion}} {{ip}} -sV -Pn" on ip 172.16.209.85
Depending on the complexity of the script, results may take some time to appear.
[~] Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-11 15:02 IST
NSE: Loaded 47 scripts for scanning.
Initiating ARP Ping Scan at 15:02
Scanning 172.16.209.85 [1 port]
Completed ARP Ping Scan at 15:02, 0.08s elapsed (1 total hosts)
Initiating Parallel DNS resolution of 1 host. at 15:02
Completed Parallel DNS resolution of 1 host. at 15:02, 0.02s elapsed
DNS resolution of 1 IPs took 0.02s. Mode: Async [#: 3, OK: 0, NX: 1, DR: 0, SF: 0, TR: 1, CN: 0]
Initiating SYN Stealth Scan at 15:02
Scanning 172.16.209.85 [1 port]
Discovered open port 22/tcp on 172.16.209.85
Completed SYN Stealth Scan at 15:02, 0.01s elapsed (1 total ports)
Initiating Service scan at 15:02
Scanning 1 service on 172.16.209.85
Completed Service scan at 15:02, 0.07s elapsed (1 service on 1 host)
NSE: Script scanning 172.16.209.85.
NSE: Starting runlevel 1 (of 2) scan.
Initiating NSE at 15:02
Completed NSE at 15:02, 0.01s elapsed
NSE: Starting runlevel 2 (of 2) scan.
Initiating NSE at 15:02
Completed NSE at 15:02, 0.00s elapsed
Nmap scan report for 172.16.209.85
Host is up, received arp-response (0.0017s latency).
Scanned at 2025-11-11 15:02:24 IST for 0s

PORT      STATE SERVICE REASON          VERSION
22/tcp    open  ssh                syn-ack ttl 64 OpenSSH 10.0p2 Debian 7~bpo12+1 (protocol 2.0)
MAC Address: 08:00:27:E7:D7:3F (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Read data files from: /usr/share/nmap
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 0.45 seconds
Raw packets sent: 2 (72B) | Rcvd: 2 (72B)

(jadhav@Kali)-[~/Downloads/RustScan-2.4.1]
```

4) hping3 — custom packet probes & timing

Purpose: Craft specific TCP flag probes (SYN, FIN, etc.), simulate unusual patterns or timed bursts.

hping3 is used to **create and send custom packets** (TCP, UDP, ICMP, RAW-IP) to test network security, detect open ports, firewall behavior, or measure response times.

Install (Kali):

```
sudo apt update
sudo apt install -y hping3
```

```
(jadhav@Kali)-[~]
$ sudo apt install -y hping3
[sudo] password for jadhav:
hping3 is already the newest version (3.a2.ds2-11-kali1).
The following packages were automatically installed and are no longer required:
  cruu      libintl-perl      libmongoc-1.0-0t64      libsort-naturally-perl      tini
  libbson-1.0-0t64  libintl-xs-perl      libmongocrypt0      needrestart
  libcompell1  libmodule-find-perl  libproc-processtable-perl  python3-pycrui
Use 'sudo apt autoremove' to remove them.
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1303
(jadhav@Kali)-[~]
$
```

simple SYN probe (4 packets)

```
sudo hping3 -S -p 22 -c 4 172.16.209.85
```

Options explained:

- **-S** : set SYN flag (like starting a TCP handshake).
- **-p 22** : destination port 22.
- **-c 4** : send 4 packets then stop.
- **172.16.209.85** : target.

What you'll see (scenario):

- Console: reply lines showing if SYN/ACK returned (open) or RST (closed).
- Pcap: SYNs and SYN/ACKs; timestamps reflect crafted timing.
- Logs: no auth lines

```
(jadhav@Kali)-[~]
$ sudo hping3 -S -p 22 -c 4 172.16.209.85
[sudo] password for jadhav:
HPING 172.16.209.85 (eth0 172.16.209.85): S set, 40 headers + 0 data bytes
len=46 ip=172.16.209.85 ttl=64 DF id=0 sport=22 flags=SA seq=0 win=64240 rtt=9.2 ms
len=46 ip=172.16.209.85 ttl=64 DF id=0 sport=22 flags=SA seq=1 win=64240 rtt=10.5 ms
len=46 ip=172.16.209.85 ttl=64 DF id=0 sport=22 flags=SA seq=2 win=64240 rtt=9.3 ms
len=46 ip=172.16.209.85 ttl=64 DF id=0 sport=22 flags=SA seq=3 win=64240 rtt=5.1 ms
— 172.16.209.85 hping statistic —
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 5.1/8.5/10.5 ms
(jadhav@Kali)-[~]
$
```

2. Brute-force / Credential stuffing (multiple failed logins)

1,crunch

crunch is a **command-line wordlist generator** that **builds every possible string from a charset you choose, optionally following a pattern**. It's shipped with Kali and Debian-based distros and is commonly used to create wordlists for testing in a controlled lab. ([Kali Linux](#))

Installation

On Debian / Kali:

```
sudo apt update
sudo apt install crunch
```

```
(jadhav@Kali)-[~]
$ sudo apt install crunch
[sudo] password for jadhav:
crunch is already the newest version (3.6-3.1).
crunch set to manually installed.
The following packages were automatically installed and are no longer required:
  criu      libintl-perl      libmongoc-1.0-0t64  libsort-naturally-perl  tini
  libbson-1.0-0t64  libintl-xs-perl    libmongocrypt0      needrestart
  libcompel1      libmodule-find-perl  libproc-processtable-perl  python3-pycriu
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1306
(jadhav@Kali)-[~]
$
```

Basic syntax

`crunch <min-len> <max-len> [charset] [options]`

- **<min-len> / <max-len>** — required, smallest and largest generated length.
- **[charset]** — list of characters to use (e.g. `abc123` or `abcdefghijklmnopqrstuvwxyz`). If omitted you can use `-f` with a charset file.

Common flags & features (most useful)

- **-o <file>** — write output to **<file>** (or use `START` to write chunked files).
- **-o-** — write to stdout (useful to pipe into other tools).
- **-t <pattern>** — pattern mode: embed literals and placeholders (see placeholders below). Pattern length must match total length requested.
- **-f <path/to/charset.lst> <name>** — use a named charset from a charset file (useful for custom sets).
- **-s <start-string>** — start at a specific wordlist entry (resume from a given point).
- **-z <gzip|bzip2|lzma>** — compress chunked output (used with `-o START`).

- **-b <size>** — create chunked output files of maximum size (e.g., **-b 10M**), useful to avoid enormous single files.
- **-i** — invert order of generation (rarely needed).
- **-l** — when using **-t**, treat placeholders as literals (helpful if a placeholder char is part of the literal).

Placeholders in **-t** pattern mode

-t lets you mix literal text with placeholders. Common placeholder symbols (typical mapping — check **crunch -h** on your system to confirm your version):

- **@** = lowercase letters (a–z)
- **,** = UPPERCASE letters (A–Z)
- **%** = digits (0–9)
- **^** = symbols (special chars)

Example: **-t TestPass%%%** (with length 11) will produce **TestPass000** → **TestPass999**.

Quick workflow

First create a test user in target machine which is we are using parrot

```
[x]--[jadhav@parrot]--[~]
-- $sudo adduser user
Adding user `user' ...
Adding new group `user' (1006) ...
Adding new user `user' (1006) with group `user (1006)' ...
Creating home directory `/home/user' ...
Copying files from `/etc/skel' ...
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for user
Enter the new value, or press ENTER for the default
01216 Full Name []:
Room Number []:
Work Phone []:
Home Phone []:
Other []:
Is the information correct? [Y/n]
Adding new user `user' to supplemental / extra groups `users' ...
Adding user `user' to group `users' ...
[jadhav@parrot]--[~]
-- $
```

Here we give the password is **abc1**

Now in attacker machine :

1. Make directory:

```
(jadhav@Kali)-[~]  
$ cd Desktop/  
  
(jadhav@Kali)-[~/Desktop]  
$ mkdir wordlists
```

2. Make a tiny test list:

```
(jadhav@Kali)-[~/Desktop/wordlists]  
$ pwd  
/home/jadhav/Desktop/wordlists
```

crunch 4 4 ac1b > /home/jadhav/Desktop/wordlists/pass.txt

```
(jadhav@Kali)-[~/Desktop/wordlists]  
$ crunch 4 4 ac1b > /home/jadhav/Desktop/wordlists/pass.txt  
Crunch will now generate the following amount of data: 1280 bytes  
0 MB  
0 GB  
0 TB  
0 PB  
Crunch will now generate the following number of lines: 256
```

Cat pass.txt

```
(jadhav@Kali)-[~/Desktop/wordlists]  
$ cat pass.txt  
aaaa  
aaac  
aaa1  
aaab
```

```
abc1  
abcb  
ab1a
```

2,Hydra

hydra (THC-Hydra) is a fast, **parallelized network login cracker (dictionary + brute-force)** that supports many protocols (SSH, FTP, HTTP forms, RDP, SMB, VNC, SMTP, POP3, IMAP, etc.). It's intended for security testing by researchers/pen testers.

Install

On Kali / Debian:

sudo apt update

sudo apt install hydra

```
(jadhav@Kali)-[~]
$ sudo apt install hydra
[sudo] password for jadhav:
hydra is already the newest version (9.6-1).
The following packages were automatically installed and are no longer required:
  criu          libintl-perl      libmongoc-1.0-0t64  libsort-naturally-perl  tini
  libbson-1.0-0t64  libintl-xs-perl    libmongocrypt0       needrestart
  libcompel1       libmodule-find-perl  libproc-processtable-perl  python3-pycru
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1306
(jadhav@Kali)-[~]
$
```

Core command syntax

hydra [options] target service [service-options]

Minimal example (single user, password list, SSH):

hydra -l user -P /home/jadhav/Desktop/wordlists/pass.txt ssh://172.16.209.85

```
(jadhav@Kali)-[~/Desktop/wordlists]
$ hydra -l user -P /home/jadhav/Desktop/wordlists/pass.txt ssh://172.16.209.85
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-11-11 19:35:05
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 16 tasks per 1 server, overall 16 tasks, 256 login tries (l:1/p:256), ~16 tries per task
[DATA] attacking ssh://172.16.209.85:22/
[22][ssh] host: 172.16.209.85 login: user password: abc1
1 of 1 target successfully completed, 1 valid password found
[WARNING] Writing restore file because 3 final worker threads did not complete until end.
[ERROR] 3 targets did not resolve or could not be connected
[ERROR] 0 target did not complete
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2025-11-11 19:35:17
(jadhav@Kali)-[~/Desktop/wordlists]
$
```

Most important options

- `-l LOGIN` : single username.
- `-L FILE` : file with usernames (one per line).
- `-p PASS` : single password.
- `-P FILE` : file with passwords (one per line).
- `-t TASKS` : number of parallel connections (threads). Default often 16 — increase carefully.
- `-s PORT` : use non-default port.
- `-S` : connect via SSL/TLS (e.g., `ftps`, `https` modules).
- `-e nsr` : extra checks: `n` = try empty password, `s` = try password = username, `r` = try reverse (username as password).
- `-f` : exit after first found pair (per host).
- `-F` : exit after first found overall when using `-M` (multi-host).
- `-M FILE` : attack many hosts (file with targets).
- `-o FILE` : write found credentials to FILE. `-b FORMAT` chooses file format (text/json).
- `-x min:max:charset` : instruct hydra to brute-force password generation (min/max length, charset spec).
- `-R` : restore a previously aborted session (uses `hydra.restore`).
- `-w TIME` / `-W TIME` : set timeouts / per-connection wait pauses (useful to avoid lockouts).
- `-v` / `-V` : verbosity; `-V` prints each tried login:pass.
- `-u` : rotate through usernames (try password1 on all users, then password2, ...) — useful to avoid account lockouts.
- `-I` : ignore existing restore file.

3,CUPP

CUPP (Common User Passwords Profiler) is a small password-wordlist generator that builds targeted wordlists from personal information about a user (names, birthdays, pets, hobbies, patterns). It's designed to help penetration testers create focused wordlists for social-engineering style password cracking — much smaller and more likely to contain a target's real password than huge generic lists.

Target setup:

Create a user and password based on this ((names, birthdays, pets, hobbies, patterns))

Username: user1

Password: user12004(username+birthyear)

```
[jadhav@parrot]-[~]  
$sudo adduser user1  
[sudo] password for jadhav:  
Adding user `user1' ...  
Adding new group `user1' (1007) ...  
Adding new user `user1' (1007) with group `user1 (1007)' ...  
Creating home directory `/home/user1' ...  
Copying files from `/etc/skel' ...  
New password:  
Retype new password:  
No password has been supplied.  
New password:  
Retype new password:  
passwd: password updated successfully
```

Install

CUPP is usually a simple Python script you can clone and run:

install prerequisites (if needed)

sudo apt update

sudo apt install -y git python3 python3-pip

```
(jadhav@Kali)-[~]
$ sudo apt install -y git python3 python3-pip
[sudo] password for jadhav:
python3 is already the newest version (3.13.7-1).
python3 set to manually installed.
python3-pip is already the newest version (25.2+dfsg-1).
The following packages were automatically installed and are no longer required:
  criu      libintl-perl      libmongoc-1.0-0t64      libsort-naturally-perl      tini
  libbson-1.0-0t64  libintl-xs-perl      libmongocrypt0      needrestart
  libcompell      libmodule-find-perl  libproc-processtable-perl  python3-pycriu
Use 'sudo apt autoremove' to remove them.

Upgrading:
  git  git-man

Summary:
  Upgrading: 2, Installing: 0, Removing: 0, Not Upgrading: 1304
  Download size: 11.5 MB
  Space needed: 173 kB / 26.3 GB available

Get:1 http://kali.download/kali kali-rolling/main amd64 git amd64 1:2.51.0-1 [9,259 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 git-man all 1:2.51.0-1 [2,286 kB]
Fetched 11.5 MB in 2s (6,097 kB/s)
(Reading database ... 426016 files and directories currently installed.)
Preparing to unpack .../git_1%3a2.51.0-1_amd64.deb ...
Unpacking git (1:2.51.0-1) over (1:2.50.1-0.1) ...
Preparing to unpack .../git-man_1%3a2.51.0-1_all.deb ...
Unpacking git-man (1:2.51.0-1) over (1:2.50.1-0.1) ...
Setting up git-man (1:2.51.0-1) ...
Setting up git (1:2.51.0-1) ...
Processing triggers for man-db (2.13.1-1) ...
Processing triggers for kali-menu (2025.3.0) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.
No services need to be restarted.
No containers need to be restarted.
No user sessions are running outdated binaries.
No VM guests are running outdated hypervisor (qemu) binaries on this host.
```

clone the repo

git clone <https://github.com/Mebus/cupp.git>

```
(jadhav@Kali)-[~]
$ git clone https://github.com/Mebus/cupp.git
Cloning into 'cupp'...
remote: Enumerating objects: 237, done.
remote: Counting objects: 100% (79/79), done.
remote: Compressing objects: 100% (12/12), done.
remote: Total 237 (delta 70), reused 67 (delta 67), pack-reused 158 (from 1)
Receiving objects: 100% (237/237), 2.14 MiB | 446.00 KiB/s, done.
Resolving deltas: 100% (128/128), done.
Host is up (0.0056s latency).
(jadhav@Kali)-[~] tcp ports (reset)
```

cd cupp

```
(jadhav@Kali)-[~]
$ cd cupp
Host is up (0.0056s latency).
(jadhav@Kali)-[~/cupp] ports (reset)
$
```

run interactively (the script is cupp.py)

python3 cupp.py -h # show help / confirm it runs

```
usage: cupp.py [-h] [-i | -w FILENAME | -l | -a | -v] [-q]
Common User Passwords Profiler
options:
  -h, --help            show this help message and exit
  -i, --interactive      Interactive questions for user password profiling
  -w FILENAME            Use this option to improve existing dictionary, or WyD.pl output to make some pwnsauce
  -l                    Download huge wordlists from repository
  -a                    Parse default usernames and passwords directly from Alecto DB. Project Alecto uses purified
                        databases of Phenoelit and CIRT which were merged and enhanced
  -v, --version          Show the version of this program.
  -q, --quiet           Quiet mode (don't print banner)

(jadhav@Kali)-[~/cupp] ports (reset)
$
```

How to run

1. Interactive mode (recommended for targeted lists)

```
python3 -W ignore cupp.py -i
```

CUPP will prompt you for information about the target:

- real name, nickname(s)
- birthdate(s)
- partner/children/pets names
- hobbies, favorite teams, phone numbers, important years
- special patterns (e.g., appending 123, use of !, common substitutions)

At the end it asks where to save the wordlist (give a path like `~/wordlists/cuplist_target.txt`). The result is a focused wordlist containing permutations and common variations of the supplied data.

What to enter

The better the profile data, the more effective (and smaller) the list:

- full name, nicknames, initials
- spouse/partner/kids/pet names (and short forms)
- birthdays, anniversaries, memorable years
- favorite sports team, band, hobbies (words they often use)
- aliases, email prefixes, phone suffixes
- common substitutions they use (e → 3, a → @, s → \$), and whether they append years or special chars
- typical password patterns (e.g., baseword + 123, or Year at end)

CUPP will combine these elements in many useful ways (concatenations, dates, basic substitutions, common suffixes/prefixes).

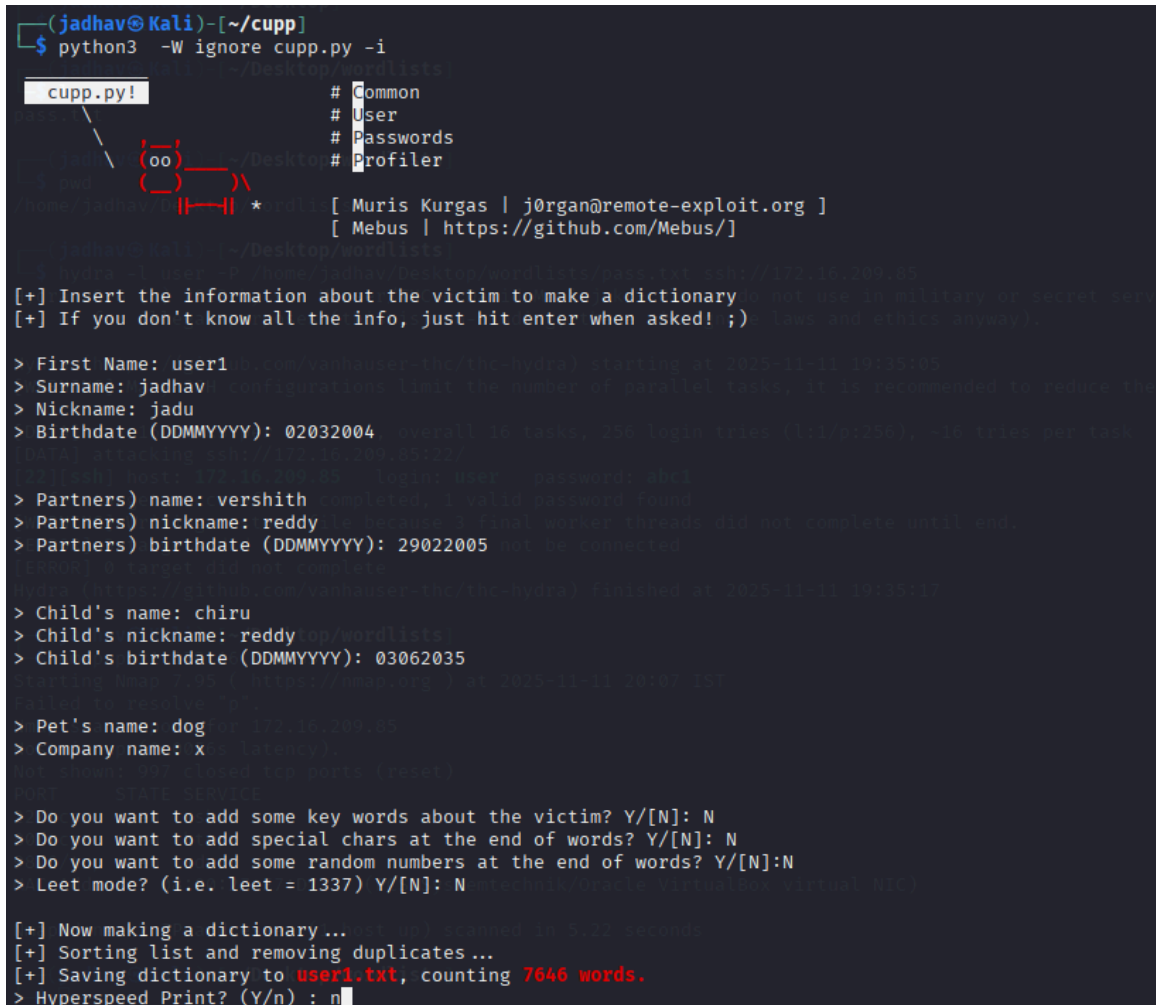
Example workflow (create list + use with hydra)

1. Generate targeted list:

```
mkdir -p ~/wordlists
```

```
cd cupp
```

```
python3 -W ignore cupp.py -i
```



```
(jadhav@Kali)-[~/cupp]
$ python3 -W ignore cupp.py -i
cupp.py! # Common
          # User
          # Passwords
          # Profiler
$ cat ~/Desktop/wordlists
[ Muris Kurgas | j0rgan@remote-exploit.org ]
[ Mebus | https://github.com/Mebus/]
jadhav@Kali: ~/Desktop/wordlists
$ cat ~/Desktop/wordlists
[+] Insert the information about the victim to make a dictionary (notice in military or secret serv
[+] If you don't know all the info, just hit enter when asked! ;) (aws and ethics anyway!).

> First Name: user1
> Surname: jadhav
> Nickname: jadu
> Birthdate (DDMMYYYY): 02032004
> Partners) name: vershith
> Partners) nickname: reddy
> Partners) birthdate (DDMMYYYY): 29022005
> Child's name: chiru
> Child's nickname: reddy
> Child's birthdate (DDMMYYYY): 03062035
> Pet's name: dog
> Company name: x
> Do you want to add some key words about the victim? Y/[N]: N
> Do you want to add special chars at the end of words? Y/[N]: N
> Do you want to add some random numbers at the end of words? Y/[N]:N
> Leet mode? (i.e. leet = 1337) Y/[N]: N
[+] Now making a dictionary...
[+] Sorting list and removing duplicates...
[+] Saving dictionary to user1.txt, counting 7646 words.
> Hyperspeed Print? (Y/n) : n
```

We got the file user1.txt

If we want to see that
Cat user1.txt

```
Chiru03 ...
Chiru03035 ...
Chiru030353 Call: ~/Desktop/wordlists/
Chiru030356 ...
Chiru0306 ...
Chiru03063 ...
Chiru030635 Call: ~/Desktop/wordlists/
Chiru03066 ...
Chiru032035 ~/Desktop/wordlists/
Chiru033 ...
Chiru033035 Call: ~/Desktop/wordlists/
Chiru03306 user -P /home/jadhav/Desktop/wordlists/pass.txt ssh://172.16.209.85
Chiru03335 (c) 2023 by van Hauser/TBC & David Maciejak - Please do not use in military or secret service organiza
Chiru0335 Illegal purposes (this is non-binding, these *t* ignore laws and ethics anyway!).
Chiru033506 ...
Chiru03353 https://github.com/vanhauser-thc/thc-hydras starting at 2023-11-11 19:35:05
Chiru03356 Long SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks; use
Chiru0336 ...
Chiru035 ... 16 tasks per 1 server, overall 16 tasks, 256 login tries (16 pr 256), -16 tries per task
Chiru03503 Using ssh://172.16.209.85:22/
Chiru035033 ssh://172.16.209.85/ login: user - passwords: abcd
Chiru035036 successfully completed, 1 valid password found
```

hydra -l user -P /home/jadhav/cupp/user1.txt ssh://172.16.209.85 -t 1 -f -vV

4,CeWL

CeWL (pronounced “cool”) is a *custom wordlist generator* written in Ruby.

It crawls a **target website** and extracts words (like names, keywords, product names, emails) from the web pages — then builds a wordlist from that real content.

In short:

CeWL = “Web crawler + wordlist generator” used to create **site-specific dictionaries** for password testing or social-engineering research.

Installation (on Kali/Linux)

CeWL comes preinstalled in Kali Linux.

If not, install manually:

sudo apt update

sudo apt install cewl

```
(jadhav@Kali)-[~] 16:209:85 ~ login: user1 - pass: "0432004" - 186 of 7646 [child 0] (0
$ sudo apt install cewl 189:85 ~ login: user1 - pass: "0432004" - 187 of 7646 [child 0] (0
[sudo] password for jadhav: 189:85 ~ login: user1 - pass: "0432004" - 188 of 7646 [child 0] (0
cewl is already the newest version (6.2.1-1). 189:85 ~ login: user1 - pass: "0500502" - 189 of 7646 [child 0] (0
cewl set to manually installed. 189:85 ~ login: user1 - pass: "0500502" - 190 of 7646 [child 0] (0
The following packages were automatically installed and are no longer required: 189:85 ~ login: user1 - pass: "0500502" - 191 of 7646 [child 0] (0
  criu libintl-xs-perl libproc-processtable-perl tini 192 of 7646 [child 0] (0
  libbson-1.0-0t64 libmodule-find-perl libsort-naturally-perl 193 of 7646 [child 0] (0
  libcompell libmongoc-1.0-0t64 needrestart 194 of 7646 [child 0] (0
  libintl-perl libmongocrypt0 python3-pycriu 195 of 7646 [child 0] (0
Use 'sudo apt autoremove' to remove them. 189:85 ~ login: user1 - pass: "0502205" - 196 of 7646 [child 0] (0
Summary: 189:85 ~ login: user1 - pass: "052005" - 197 of 7646 [child 0] (0
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1304 198 of 7646 [child 0] (0
(jadhav@Kali)-[~] 16:209:85 ~ login: user1 - pass: "0520052" - 199 of 7646 [child 0] (0
$ 16:209:85 ~ login: user1 - pass: "05200529" - 200 of 7646 [child 0] (0
```

Check version:

cewl --version

```
(jadhav@Kali)-[~]
$ cewl --version
CeWL 6.2.1 (More Fixes) Robin Wood (robin@diginiinja) (https://diginiinja/)
/usr/bin/cewl: unrecognized option '--version'
Usage: cewl [OPTIONS] ... <url>

OPTIONS:
  -h, --help: Show help.
  -k, --keep: Keep the downloaded file.
  -d <x>, --depth <x>: Depth to spider to, default 2.
  -m, --min_word_length: Minimum word length, default 3.
  -x, --max_word_length: Maximum word length, default unset.
  -o, --offsite: Let the spider visit other sites.
  --exclude: A file containing a list of paths to exclude
  --allowed: A regex pattern that path must match to be followed
  -w, --write: Write the output to the file.
  -u, --ua <agent>: User agent to send.
  -n, --no-words: Don't output the wordlist.
  -g <x>, --groups <x>: Return groups of words as well
  --lowercase: Lowercase all parsed words
  --with-numbers: Accept words with numbers in as well as just letters
  --convert-umlauts: Convert common ISO-8859-1 (Latin-1) umlauts (ä-ae, ö-oe, ü-ue, ß-ss)
  -a, --meta: include meta data.
  --meta_file file: Output file for meta data.
  -e, --email: Include email addresses.
  --email_file <file>: Output file for email addresses.
  --meta-temp-dir <dir>: The temporary directory used by exiftool when parsing files, default /tmp.
  -c, --count: Show the count for each word found.
  -v, --verbose: Verbose.
  --debug: Extra debug information.

Authentication
  --auth_type: Digest or basic.
  --auth_user: Authentication username.
  --auth_pass: Authentication password.

Proxy Support
  --proxy_host: Proxy host.
  --proxy_port: Proxy port, default 8080.
  --proxy_username: Username for proxy, if required.
  --proxy_password: Password for proxy, if required.

Headers
  --header, -H: In format name:value - can pass multiple.

<url>: The site to spider.

(jadhav@Kali)-[~]
```

If it prints version info — you're ready

Basic Syntax

cewl [options] URL

Example:

cewl <https://example.com>

cewl https://iitkottayam.ac.in/

This prints all unique words (default min length 3) found on the site to the terminal.

Option	Description
<code>-w FILE</code>	Write output to a wordlist file
<code>-m NUM</code>	Minimum word length (default 3)
<code>-d DEPTH</code>	Depth of crawl (how deep to go into linked pages)
<code>-e</code>	Include emails found on the site
<code>--meta</code>	Extract metadata (like Author, Company, etc. from PDFs, docs, etc.)
<code>-n</code>	Don't output wordlist (used when only collecting meta)
<code>-v</code>	Verbose — shows crawling progress
<code>-a</code>	Include external links (default = off, so it stays inside the same domain)
<code>-o</code>	Obey robots.txt (optional, ethical crawling)
<code>--ua "User-Agent"</code>	Custom user-agent string
<code>--with-numbers</code>	Keep words that contain numbers (like "Pass123")

Basic crawl and save words

cd Desktop/

```
(jadhav@Kali)-[~]  
$ cd Desktop/
```

Mkdir wordlist_cool

```
(jadhav@Kali)-[~/Desktop]  
$ mkdir wordlist_cool  
  
(jadhav@Kali)-[~/Desktop]  
$ pwd  
/home/jadhav/Desktop  
  
(jadhav@Kali)-[~/Desktop]  
$ cd wordlist_cool/  
  
(jadhav@Kali)-[~/Desktop/wordlist_cool]  
$ pwd  
/home/jadhav/Desktop/wordlist_cool
```

cewl https://example.com -w ~/wordlists/example_cewl.txt

Or

cewl https://iitkottayam.ac.in/ -w /home/jadhav/Desktop/wordlist_cool/pass_cewl.txt

```
(jadhav@Kali)-[~/Desktop/wordlist_cool]  
$ cewl https://iitkottayam.ac.in/ -w /home/jadhav/Desktop/wordlist_cool/pass_cewl.txt  
CewL 6.2.1 (More Fixes) Robin Wood (robin@diginiinja) (https://diginiinja/)
```

Output:

```
(jadhav@Kali)-[~/Desktop/wordlist_cool]  
$ ls  
pass_cewl.txt  
  
(jadhav@Kali)-[~/Desktop/wordlist_cool]  
$ nano pass_cewl.txt  
  
(jadhav@Kali)-[~/Desktop/wordlist_cool]
```

- Crawls the site
- Saves unique words ≥ 3

```
GNU nano 8.4 pass_cewl.txt  
iitkottayam  
research  
iit  
club  
international  
down  
prop  
security  
student  
sports  
institute  
tenders  
papers  
oga  
or  
technology  
faculty  
technical  
students  
cyber  
information  
the
```

Crawl deeper and filter word length

`cewl https://example.com -d 2 -m 5 -w ~/wordlists/deep_cewl.txt`

- `-d 2` → goes 2 links deep
- `-m 5` → only words ≥ 5 letters

Useful when target has nested pages.

Collect emails

`cewl https://example.com -e -w ~/wordlists/emails.txt`

This extracts any visible email addresses found on the site.

```
(jadhav@Kali)-[~/Desktop/wordlist_cool]
$ cewl https://iitkottayam.ac.in/ -e -w /home/jadhav/Desktop/wordlist_cool/emails_cewl.txt
CeWL 6.2.1 (More Fixes) Robin Wood (robin@diginiinja) (https://diginiinja/)
Couldn't hit the site https://iitkottayam.ac.in/, moving on
```

Extract metadata (authors, orgs) from files

`cewl https://example.com --meta -w ~/wordlists/meta.txt`

This analyzes downloadable files (like PDFs or DOCX) for metadata — often reveals usernames, authors, or organizations.

```
(jadhav@Kali)-[~/Desktop/wordlist_cool]
$ cewl -v -d 2 -m 4 -w ~/Desktop/wordlist_cool/pass_cewl.txt https://iitkottayam.ac.in/ 2>&1 | tee ~/cewl_debug.log
CeWL 6.2.1 (More Fixes) Robin Wood (robin@diginiinja) (https://diginiinja/)
Starting at https://iitkottayam.ac.in/
Couldn't hit the site https://iitkottayam.ac.in/, moving on
Writing words to file
```

5, Metasploit auxiliary modules for authentication testing:

1) Overview — what these modules do

Metasploit's auxiliary `scanner/login` modules perform *authentication testing* (username/password checks) against services — SSH, FTP, SMB, HTTP forms, SMTP, IMAP, RDP, VNC, databases, etc. They are used for:

- validating credential lists (targeted pentests)
- checking for weak/default passwords
- discovering valid accounts (to be used later with exploits/post modules)

They are **not** exploit modules; they attempt logins and report successes/failures.

2) Common module paths (quick list)

Use `search type:auxiliary name:login` or `search login` in msfconsole to find all. Common ones:

- SSH: `auxiliary/scanner/ssh/ssh_login`
- FTP: `auxiliary/scanner/ftp/ftp_login`
- SMB/Windows: `auxiliary/scanner/smb/smb_login`
- HTTP basic/form: `auxiliary/scanner/http/http_login` and `auxiliary/scanner/http/http_login_html / auxiliary/scanner/http/http_login_form` (names vary by version)
- HTTP (POST form): `auxiliary/scanner/http/http_login` with `FORM` options or `auxiliary/scanner/http/http_login` + manual POST string (check module docs)
- SMTP: `auxiliary/scanner/smtp/smtp_login`
- IMAP: `auxiliary/scanner/imap/imap_login`
- POP3: `auxiliary/scanner/pop3/pop3_login`
- RDP: `auxiliary/scanner/rdp/rdp_login` (or `rdp_login`)
- VNC: `auxiliary/scanner/vnc/vnc_login`
- Telnet: `auxiliary/scanner/telnet/telnet_login`
- MSSQL: `auxiliary/scanner/mssql/mssql_login`
- MySQL: `auxiliary/scanner/mysql/mysql_login`
- PostgreSQL: `auxiliary/scanner/postgres/postgres_login`
- SNMP: `auxiliary/scanner/snmp/snmp_login` (if community/string auth)

- Generic: `auxiliary/scanner/cred_*` or `auxiliary/scanner/*_login` (many service specific modules)

(If a module name differs on your install, `search login` will show the exact path.)

3) Basic usage pattern (one-off)

Open `msfconsole` and run a scanner module like this:

```
(jadhav@Kali)-[~]
$ msfconsole
Metasploit tip: Start commands with a space to avoid saving them to history

[Metasploit v6.4.69-dev]
+ -- --[ 2529 exploits - 1302 auxiliary - 432 post
+ -- --[ 1672 payloads - 49 encoders - 13 nops
+ -- --[ 9 evasion

Metasploit Documentation: https://docs.metasploit.com/
msf6 > 
```

If you do `search login`

`msf6 > use auxiliary/scanner/ssh/ssh_login`

```
msf6 > use auxiliary/scanner/ssh/ssh_login
msf6 auxiliary(scanner/ssh/ssh_login) > show options
```

`msf6 auxiliary(ssh_login) > show options`


```
msf6 auxiliary(scanner/ssh/ssh_login) > show options

Module options (auxiliary/scanner/ssh/ssh_login):
```

Name	Current Setting	Required	Description
ANONYMOUS_LOGIN	false	yes	Attempt to login with a blank username and password
BLANK_PASSWORDS	false	no	Try blank passwords for all users
BRUTEFORCE_SPEED	5	yes	How fast to bruteforce, from 0 to 5
CreateSession	true	no	Create a new session for every successful login
DB_ALL_CREDS	false	no	Try each user/password couple stored in the current database
DB_ALL_PASS	false	no	Add all passwords in the current database to the list
DB_ALL_USERS	false	no	Add all users in the current database to the list
DB_SKIP_EXISTING	none	no	Skip existing credentials stored in the current database (Acceptable values: none, user, user@realm)
PASSWORD		no	A specific password to authenticate with
PASS_FILE		no	File containing passwords, one per line
RHOSTS		yes	The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT	22	yes	The target port
STOP_ON_SUCCESS	false	yes	Stop guessing when a credential works for a host
THREADS	1	yes	The number of concurrent threads (max one per host)
USERNAME		no	A specific username to authenticate as
USERPASS_FILE		no	File containing users and passwords separated by space, one pair per line
USER_AS_PASS	false	no	Try the username as the password for all users
USER_FILE		no	File containing usernames, one per line
VERBOSE	false	yes	Whether to print output for all attempts

View the full module info with the `info`, or `info -d` command.

msf6 auxiliary(ssh_login) > set RHOSTS 172.16.209.85 # single host or CIDR/range

```
msf6 auxiliary(scanner/ssh/ssh_login) > set RHOSTS 172.16.209.85
RHOSTS => 172.16.209.85
msf6 auxiliary(scanner/ssh/ssh_login) >
```

msf6 auxiliary(ssh_login) > set USER_FILE /root/users.txt # or set USERNAME alice
 msf6 auxiliary(ssh_login) > set PASS_FILE /root/pass.txt # or set PASSWORD 'summer2025'

```
msf6 auxiliary(scanner/ssh/ssh_login) > set USERNAME user1
USERNAME => user1
msf6 auxiliary(scanner/ssh/ssh_login) > set PASS_FILE user12004
PASS_FILE => user12004
msf6 auxiliary(scanner/ssh/ssh_login) >
```

Two ways to supply credentials

You can give the module either a single value or a file of values.

Single username:

```
set USERNAME alice
```

Username file (one username per line):

```
set USER_FILE /root/users.txt
```

Single password:

```
set PASSWORD 'summer2025'
```

Password file (one password per line):

```
set PASS_FILE /root/pass.txt
```

msf6 auxiliary(ssh_login) > set THREADS 4

be conservative

msf6 auxiliary(ssh_login) > run

```
msf6 > use auxiliary/scanner/ssh/ssh_login
msf6 auxiliary(scanner/ssh/ssh_login) > set RHOSTS 172.16.209.85
RHOSTS => 172.16.209.85
msf6 auxiliary(scanner/ssh/ssh_login) > set USERNAME user1
USERNAME => user1
msf6 auxiliary(scanner/ssh/ssh_login) > set PASSWORD 'user12004'
PASSWORD => user12004
msf6 auxiliary(scanner/ssh/ssh_login) > set THREADS 4
THREADS => 4
msf6 auxiliary(scanner/ssh/ssh_login) > run
[*] 172.16.209.85:22 - Starting bruteforce
[+] 172.16.209.85:22 - Success: 'user1:user12004' 'uid=1007(user1) gid=1007(user1) groups=1007(user1),100(users) Lin
ux parrot 6.12.32-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.32-1parrot1 (2025-06-27) x86_64 GNU/Linux '
[*] SSH session 1 opened (172.16.223.193:46515 -> 172.16.209.85:22) at 2025-11-11 22:12:06 +0530
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf6 auxiliary(scanner/ssh/ssh_login) >
```

Output will show successful logins. Metasploit also stores credentials in the **creds** database table and writes loot.

For user:

```
(root@Kali)-[/home/jadhav]
# # make sure directory exists
sudo mkdir -p /root/wordlists

# create the file with newlines (good for single-line entry lists)
sudo printf "testuser\nuser1\nbob\n" > /root/wordlists/users.txt

(root@Kali)-[/home/jadhav]
#
```

For pass

```

(root@Kali)-[/home/jadhav]
# # make dir if needed
sudo mkdir -p /root/wordlists

# create password file with newline-separated entries (safe & simple)
sudo printf "password\n user12004\nsummer2025\n" > /root/wordlists/pass.txt

# secure the file so only root (or your msf user) can read it
sudo chmod 600 /root/wordlists/pass.txt

# verify contents
sudo head -n 20 /root/wordlists/pass.txt
ls -l /root/wordlists/pass.txt
password
user12004
summer2025
-rw-r--r-- 1 root root 31 Nov 11 22:19 /root/wordlists/pass.txt

(root@Kali)-[/home/jadhav]
#

```

If we want to add list of username and passwords

This is the way:

```

msf6 > use auxiliary/scanner/ssh/ssh_login
msf6 auxiliary(scanner/ssh/ssh_login) > set RHOSTS 172.16.209.85
RHOSTS => 172.16.209.85
msf6 auxiliary(scanner/ssh/ssh_login) > set USER_FILE /root/wordlists/users.txt
USER_FILE => /root/wordlists/users.txt
msf6 auxiliary(scanner/ssh/ssh_login) > set PASS_FILE /root/wordlists/pass.txt
PASS_FILE => /root/wordlists/pass.txt
msf6 auxiliary(scanner/ssh/ssh_login) > set THREADS 4
THREADS => 4
msf6 auxiliary(scanner/ssh/ssh_login) > run
[-] Msf::OptionValidateError One or more options failed to validate: USER_FILE, PASS_FILE.
msf6 auxiliary(scanner/ssh/ssh_login) >

```

4) Example: test SSH with a single user and password list

```
use auxiliary/scanner/ssh/ssh_login
set RHOSTS 172.16.209.85
set USERNAME testuser
set PASS_FILE /root/wordlists/crunch_small.txt
set THREADS 2
set StopOnSuccess true    # module-specific option in some modules; stops after first success
run
```

If `StopOnSuccess` isn't available for that module, use `-j` to background and monitor results. Always check `show options` for module-specific flags.

5) Example: test SMB/Windows logins with username list + password list

```
use auxiliary/scanner/smb/smb_login
set RHOSTS 10.0.0.0/24
set SMBUserFile ~/wordlists/users.txt # or set USER_FILE
set PASS_FILE ~/wordlists/target_pass.txt
set THREADS 8
run
```

SMB modules often accept domain/workgroup options (e.g., `SMBDomain`).

6) HTTP form (POST) auth testing

HTTP auth requires the correct POST pattern and failure string. Example (pattern syntax varies; read module docs):

```
use auxiliary/scanner/http/http_login
set RHOSTS web.target.local
set RPORT 80
set TARGETURI /login.php
```

```
set USER_FILE ~/wordlists/users.txt
set PASS_FILE ~/wordlists/passwords.txt
set FORM "username=^USER^&password=^PASS^"
set FAIL_REGEX "Invalid username or password" # string that indicates failed login
set THREADS 4
run
```

If your module requires a different option name (`POSTDATA`, `USERPASS` etc.), use `show options` to see exact fields.

7) Using RHOSTS ranges and db integration

You can scan multiple hosts with `RHOSTS` (CIDR/range). It's good to import target discovery into Metasploit DB first:

```
db_nmap -sS -p 22,80,139,445 10.0.0.0/24
# then inside msfconsole:
use auxiliary/scanner/ssh/ssh_login
set RHOSTS 10.0.0.0/24
set USER_FILE users.txt
set PASS_FILE passes.txt
run
```

Storing results in the DB lets you view `services`, `hosts`, and `creds` later:

```
hosts
services
creds
```

3. Successful access (one accepted login)

Purpose

Simulate one valid SSH login to produce an `Accepted password` event and an established SSH session (so you can later simulate/observe file transfer).

Prep — quick checks (do these first)

- Ensure `testuser` exists on **Parrot** and you know the test password (e.g., `TestPass123!`).
- Keep the lab isolated (no Internet).
- Start packet capture before you try the login:

```
sudo tcpdump -nn -tttt -i eth0 "host 172.16.223.193 and host 172.16.209.85 and tcp port 22" -w ~/ssh_success.pcap
```

Explain: `-nn` numeric, `-tttt` human timestamps, `-i eth0` interface, filter only traffic between attacker & target on port 22, `-w` write file.

1) Simple interactive SSH (manual)

```
ssh user1@172.16.209.85
```

```
# then type password: user12004
```

What it does / notes:

- Uses the system `ssh` client; interactive login.
- Useful to visibly confirm a successful session.

- [illegible]